

typesafe-ai/jev x Python 標準ライブラリ

話題の Jev、 使ってみた

文章を書かない AI を、Python だけで叩く。

kanaharu20

Kanamitsu Haruya — 42 intra: hkanamit

github.com/kanaharu20/jev-learn

```
> python3 01_hello.py
```

```
evaluate(  
  state="サポート担当者は顧客に全額返金を実施した。",  
  questions={  
    "refunded": {"type": "boolean",  
                  "instructions": "返金が行われたか?"},  
  },  
)  
# → {"refunded": {"type": "boolean", "probability": 0.96}}
```

WHAT

Jev は生成しない。判断して、確率を返す

```
> request - POST /v4/ai/evaluation-model  
{  
  "state": "サポート担当者は顧客に全額返金を実施した。",  
  "questions": {  
    "refunded": {  
      "type": "boolean",  
      "instructions": "返金が行われたか?"  
    }  
  }  
}
```



```
> response  
{  
  "answers": {  
    "refunded": {  
      "type": "boolean",  
      "probability": 0.96  
    }  
  },  
  "usage": {"inputTokens": 297, "outputTokens": 21}  
}
```

状況と型付きの質問を渡す

判断結果のみが返る

ROADMAP

4 本のスクリプトで、01 から順に

01

01_hello.py

boolean を 1 問

70 行

02

02_three_types.py

3 つの型を一度に

34 行

03

03_agent_decision.py

判断させて分岐する

61 行

04

04_vs_llm.py

Claude Haiku と比べる

20 行

4 本 + jev.py で **272** 行。標準ライブラリのみ。

STEP 00 • jev.py

API キーは Keychain から読む

```
> jev.py --load_api_key()

1 def load_api_key():
2     """環境変数にあればそれを、なければ macOS の Keychain から読む。"""
3     key = os.environ.get("AI_GATEWAY_API_KEY")
4     if not key:
5         try:
6             key = subprocess.run(
7                 ["/usr/bin/security", "find-generic-password",
8                  "-s", "Vercel AI Gateway", "-a", "vercel-ai-gateway", "-w"],
9                 capture_output=True, text=True,
10                ).stdout.strip()
11         except FileNotFoundError:
12             key = "" # macOS 以外。Keychain が無い
13     if not key:
14         raise SystemExit("API キーが見つかりません。…")
15     return key
16
17
18 API_KEY = load_api_key()
```

- 1 setup コマンドが保存してくれる
- 2 環境変数があれば優先
- 3 macOS 以外でも落ちない
- 4 `.env` を作らない

STEP 01 • 01_hello.py

HTTP を 1 本、自分で送る

```
> 01_hello.py - evaluate()

1 def evaluate(model, state, questions):
2     req = urllib.request.Request(
3         "https://ai-gateway.vercel.sh/v4/ai/evaluation-model",
4         data=json.dumps({"state": state, "questions": questions}).encode(),
5         headers={
6             "Authorization": f"Bearer {API_KEY}",
7             "Content-Type": "application/json",
8             "ai-model-id": model,
9             "ai-evaluation-model-specification-version": "4",
10            "ai-gateway-protocol-version": "0.0.1",
11            "ai-gateway-auth-method": "api-key",
12        },
13        method="POST",
14    )
15    with urllib.request.urlopen(req) as res:
16        return json.load(res)
```

- 1 専用のエンドポイント
- 2 モデルはヘッダーで指定
- 3 叩き先は **Vercel AI Gateway**。
04 の Claude Haiku も同じ経路

STEP 01 • 01_hello.py

返ってくるのは確率だけ

> 01_hello.py(長い行は折り返しあり)

```
1 result = evaluate(  
2     model="typesafe-ai/jev",  
3     state="サポート担当者は顧客に全額返金を実施した。",  
4     questions={  
5         "refunded": {"type": "boolean", "instructions": "返金が行われたか?"},  
6     },  
7 )  
8  
9 print(json.dumps(result["answers"], indent=2, ensure_ascii=False))  
10 print("usage:", result.get("usage"))  
11 print("レスポンスのキー:", list(result))
```

> \$ python3 01_hello.py

```
{  
  "refunded": {  
    "type": "boolean",  
    "probability": 0.96  
  }  
}  
usage: {'inputTokens': 297, 'outputTokens': 21}  
レスポンスのキー: ['answers', 'rounding', 'usage',  
                    'warnings', 'providerMetadata']
```

- 1 キー名は自分で決める
- 2 True / False に丸めるのは呼ぶ側
- 3 **answers** が全部ではない

STEP 02 • 02_three_types.py

3つの型を、1リクエストで

- 1 choice は **dict**
- 2 score は **list**(順番 = 段階)
- 3 呼び出しは 1 回のみ

```
> 02_three_types.py
1 result = evaluate(state, {
2     "isUrgent": {
3         "type": "boolean",
4         "instructions": "この問い合わせは緊急か?",
5     },
6     "department": {
7         "type": "choice",
8         "instructions": "どのチームが対応すべきか?",
9         "criteria": {
10             "billing": "料金・請求・返金の話",
11             "technical": "不具合や連携の問題",
12             "sales": "価格や契約の相談",
13         },
14     },
15     "frustration": {
16         "type": "score",
17         "instructions": "顧客の苛立ちの度合い",
18         "criteria": ["落ち着いている", "困っているが丁寧", "かなり怒っている"],
19     },
20 })
```

STEP 02 • 出力を読む

score 1.05 = 段階番号の期待値

> 02_three_types.py(state は折り返しあり)

```
1 state = ("3日前から Stripe 連携がずっと失敗しています。"  
2         "売上が止まっていて困っています。至急お願いします。")  
3 # result = evaluate(state, {...}) ← 前のスライド  
4  
5 conf = confidences(result) # providerMetadata から取り出す  
6  
7 for name, answer in result["answers"].items():  
8     print(f"\n{name}: {answer}")  
9     print(" confidence:", conf.get(name, "なし(boolean には付かない)"))
```

> \$ python3 02_three_types.py(整形)

```
isUrgent: {'type': 'boolean', 'probability': 0.94}
```

```
confidence: なし(boolean には付かない)
```

```
department: {'type': 'choice', 'choice': 'technical',  
             'probabilities': {'billing': 0, 'sales': 0,  
                              'technical': 1}}
```

```
confidence: 0.99
```

```
frustration: {'type': 'score', 'score': 1.05,  
              'probabilities': {'0': 0, '1': 0.95, '2': 0.05}}
```

```
confidence: 0.93
```

frustration.probabilities

0・落ち着いている 0.00



1・困っているが丁寧 0.95



2・かなり怒っている 0.05



$$0 \times 0 + 1 \times 0.95 + 2 \times 0.05 = 1.05$$

確率の合計は1。1.05 は確率ではなく
段階番号の平均なので 0～2 の値を取
る。

分布ごと返る

STEP 02・ドキュメント未掲載

confidence は answers の外にいる

> response – answers 以外の部分

```
"providerMetadata": {  
  "typesafe": {  
    "confidence": {"department": 0.99,  
                  "frustration": 0.93}  
  }  
}
```

> jev.py

```
1 def confidences(result):  
2     return (result.get("providerMetadata", {})  
3             .get("typesafe", {})  
4             .get("confidence", {}))
```

- 1 付くのは **choice** と **score** だけ
- 2 公式ドキュメントに記載なし
- 3 消えても落ちないよう `.get()` で受ける

同じ分類・曖昧さだけ変えた実測

state	最上位	confidence
明確	1.00	1.00
やや曖昧	0.58	0.37
かなり曖昧	0.51	0.26

最上位が **0.51** = 五分五分に見えても、confidence は **0.26** まで落ちている。

確率とは別物。 choice のしきい値はこちらで切る。

STEP 03 • 03_agent_decision.py

state は辞書のまま渡せる

```
> 03_agent_decision.py  
1 agent_state = {  
2     "goal": "ユーザーの注文 #1234 をキャンセルする",  
3     "toolsTried": ["lookup_order"],  
4     "lastToolResult": {"status": "shipped", "shippedAt": "2026-09-16"},  
5     "attempts": 1,  
6 }
```

```
> jev.py - 送るときは json.dumps するだけ  
{"state": state, "questions": questions}
```

- 1 キャンセルしたいのに**発送済み**
- 2 ツールの戻り値をそのまま入
れる

STEP 03 • 03_agent_decision.py

「次の一手」を choice で聞く

```
> 03_agent_decision.py
1 result = evaluate(agent_state, {
2     "nextAction": {
3         "type": "choice",
4         "instructions": "次にとるべき行動は?",
5         "criteria": {
6             "cancel_order": "キャンセル API を呼ぶ(未発送のときだけ可能)",
7             "start_return": "発送済みなので返品手続きを始める",
8             "ask_user": "ユーザーに確認が必要",
9             "give_up": "これ以上は無理。停止する",
10        },
11    },
12    "needsHuman": {
13        "type": "boolean",
14        "instructions": "人間の担当者にエスカレーションすべきか?",
15    },
16 })
17
18 answers = result["answers"]
19 conf = confidences(result) # nextAction にだけ付く
```

- 1 キー = 関数名
- 2 説明文が実質のプロンプト
- 3 判断ごとに質問を分ける
- 4 返り値を丸ごと受ける

STEP 03 • 03_agent_decision.py

判断は Jev、実行は match 文

```
> 03_agent_decision.py

1 # 迷ったまま副作用のある API を呼ばない。取れなければ 0.0 扱い。
2 if conf.get("nextAction", 0.0) < 0.5:
3     print(f"→ confidence が {conf.get('nextAction')} と低い。実行せず確認に回す")
4 else:
5     match answers["nextAction"]["choice"]:
6         case "cancel_order": print("→ cancel_order() を呼ぶ")
7         case "start_return": print("→ start_return() を呼ぶ")
8         case "ask_user":     print("→ ユーザーに質問する")
9         case "give_up":      print("→ 停止")
10
11 # boolean には confidence が付かないので probability で切る
12 if answers["needsHuman"]["probability"] > 0.7:
13     print("→ 人間に引き継ぐ")
```

nextAction.probabilities

start_return		0.94
ask_user		0.06
cancel_order		0
give_up		0

nextAction.confidence = 0.92

needsHuman.probability = 0.82

```
> $ python3 03_agent_decision.py
```

→ start_return() を呼ぶ
→ 人間に引き継ぐ

> 曖昧な state に差し替えると(抜粋)

```
confidence: {'nextAction': 0.36}
→ 実行せず確認に回す
```

迷ったら動かない。

STEP 04 • 04_vs_llm.py

同じ分類を Claude Haiku にも

- 1 同じ Gateway の chat API
- 2 LLM には「お願い」するしかない
- 3 Jev には dict を渡すだけ

```
> 04_vs_llm.py(長い行だけ折り返し)

1 state = "請求書の金額が先月の2倍になっています。理由を教えてください。"
2 categories = {"billing": "料金・請求", "technical": "不具合", "sales": "契約相談"}
3
4 # --- Jev: 型付きの答えが直接返る ---
5 t = time.time()
6 jev = evaluate(state, {
7     "category": {"type": "choice",
8                 "instructions": "問い合わせの種別は?",
9                 "criteria": categories},
10 })
11 print(f"Jev  {int((time.time() - t) * 1000)}ms →", jev["answers"]["category"])
12 print("      confidence →", confidences(jev).get("category")) # 迷っていれば下がる
13
14 # --- LLM: 文章で返るので、パースが必要 ---
15 t = time.time()
16 text = chat(
17     f"次の問い合わせを {' / '.join(categories)} のどれかに分類し、"
18     f"キー名だけを教えてください。\\n\\n{state}")
19 print(f"LLM  {int((time.time() - t) * 1000)}ms →", repr(text.strip()))
```

STEP 04 • 結果

どちらも正解。違いは速さと形

```
> $ python3 04_vs_llm.py
```

```
Jev 414ms → {'type': 'choice', 'choice': 'billing', 'probabilities': {'sales': 0, 'billing': 1, 'technical': 0}}
```

```
confidence → 1
```

```
LLM 1031ms → 'billing'
```

JEV

414_{ms}

dict + 確率。そのまま match へ
出力 **\$0**(入力 \$0.042 / 100 万)

CLAUDE HAIKU 4.5

1031_{ms}

文字列。検証は自前

計測は 1 回。参考値

共通部品・jev.py(1/2)

5xx だけ再試行する

```
> jev.py - _post()

1 def _post(path, body, extra_headers):
2     req = urllib.request.Request(
3         GATEWAY + path,
4         data=json.dumps(body).encode(),
5         headers={"Authorization": f"Bearer {API_KEY}",
6                 "Content-Type": "application/json", **extra_headers},
7         method="POST",
8     )
9     for attempt in range(4):
10         try:
11             with urllib.request.urlopen(req) as res:
12                 return json.load(res)
13         except urllib.error.HTTPError as e:
14             if e.code < 500 or attempt == 3:
15                 raise
16             time.sleep(2 ** attempt) # 5xx は 1, 2, 4 秒待って再試行
```

- 1 送信は `_post()` に 1 本化
- 2 4xx は即 `raise`
- 3 1, 2, 4 秒待つ

共通部品・jev.py(2/2)

違いはパスとヘッダーだけ

- 1 Jev はヘッダーでモデル指定
- 2 chat は OpenAI 互換
- 3 コピーすればそのまま動く

```
> jev.py(長い行は折り返しあり)

1 def evaluate(state, questions, model="typesafe-ai/jev"):
2     return _post(
3         "/v4/ai/evaluation-model",
4         {"state": state, "questions": questions},
5         {"ai-model-id": model,
6          "ai-evaluation-model-specification-version": "4",
7          "ai-gateway-protocol-version": "0.0.1",
8          "ai-gateway-auth-method": "api-key"},
9     )
10
11
12 def chat(prompt, model="anthropic/claude-haiku-4-5"):
13     res = _post(
14         "/v1/chat/completions",
15         {"model": model,
16          "messages": [{"role": "user", "content": prompt}]},
17         {},
18     )
19     return res["choices"][0]["message"]["content"]
```

NEXT

「判断だけ切り出せる」なら、Jev

BOOLEAN

はい / いいえ

緊急か？

CHOICE

有限の選択肢

どの部署か？

SCORE

段階

どのくらい深刻か？

速い・安い・パース不要

414ms / 出力 \$0 / そのまま match / 迷いは confidence

NEXT

こんなことに使えそう

CHOICE

重い LLM の前の門番

定型回答 / 安いモデル / 高いモデル / 人

BOOLEAN ×3

PR の一次チェック

テスト・説明・破壊的変更を 1 リクエストで

SCORE

大量ログの優先度付け

数千件を深刻度順に。上位だけ人が見る

CHOICE + CONFIDENCE

エージェントの行動決定

迷ったら実行せず人に回す

選択肢が限られていて、**信頼度の変数**が生きてくる。

だから速さ・出力 \$0 が効く。実行するのは自分のコード。

WRAP UP

一緒に OSS 開発でも いかがでしょうか？

勉強用に、そのままどうぞ。

github.com/kanaharu20/jev-learn

MIT License

```
> terminal  
  
git clone https://github.com/kanaharu20/jev-learn  
cd jev-learn  
npx vercel ai-gateway setup   # 初回だけ  
python3 01_hello.py
```